

RAPPORT DE PROJET

TrueTone



GROUPE HARMONIA

Maëlys GARCIE
Jad BENDALI
Elias BOINALI
Faustine DEFERT-SIMONNEAU

EPITA PROMO 2029

Table des matières

I - Présentation du groupe.....	3
II - Organisation du projet.....	4
1 - Les objectifs.....	4
2 - Planification des tâches.....	4
III - avancement réel du projet.....	6
1) Site web.....	6
2) Interface Graphique.....	7
3) Capture audio.....	9
4) Analyse fréquentielle.....	10
5) Comparaison des notes.....	11
6) Mise en commun et architecture temps réel.....	11
7) Difficultés rencontrées.....	14
IV - Bibliographie.....	17

I - Présentation du groupe

Le groupe est constitué de quatre élèves d'EPITA en seconde année de classe préparatoire. Il a été formé en prenant en compte les affinités mais également les compétences de chacun afin de pouvoir répondre au mieux à la tâche imposée. Les étudiants viennent initialement de campus différents et se sont regroupés à Lyon. Leurs noms sont Garcie Maëlys, Bendali Jad, Boinali Elias et Defert-Simonneau Faustine. Leurs tâches durant le projet sont les suivantes :

Dans ce projet, Maëlys, qui suscite un intérêt pour l'informatique depuis le collège, est chargée de développer le site web et de s'occuper de la partie traitement de signal. Cet intérêt s'est progressivement renforcé au fil de ses années de lycée, au cours desquelles elle a pu découvrir et approfondir différents aspects de ce domaine.

Jad est responsable de l'interface graphique. Son choix a été de s'investir dans cet aspect pour créer une application à la fois fluide, responsive et agréable à utiliser, en accordant autant d'importance à l'expérience utilisateur qu'au rendu visuel. Il s'intéresse particulièrement à la manière dont la technique peut rencontrer l'esthétique.

Faustine est responsable de l'importation des notes de référence d'une guitare classique/acoustique ainsi que de la comparaison entre ces notes et celles enregistrées par l'utilisateur. Ce rôle fait le lien entre toutes les parties qui sont nécessaires pour indiquer à l'utilisateur si sa note est bien accordée ou non.

Elias, pour sa part, est responsable de la capture audio. Son rôle consiste à récupérer le signal provenant du microphone en temps réel et à le transformer en données exploitables par le reste du programme. Il travaille notamment sur la gestion de flux audio, le découpage du signal en fenêtre et la mise en place d'un prétraitement afin d'assurer une meilleure qualité du signal pour les étapes d'analyse.

II - Organisation du projet

1 - Les objectifs

L'objectif principal de ce projet est d'aider les utilisateurs à accorder leurs instruments facilement et rapidement, afin qu'ils puissent consacrer un maximum de temps à la pratique de la musique. Trop souvent, l'accordage est une étape fastidieuse, surtout pour les débutants qui ne disposent pas encore d'une oreille musicale suffisamment développée pour identifier si une corde est juste ou non. TrueTone répond à ce besoin en proposant une solution intuitive et accessible.

Pour cela, nous sommes en train de mettre au point un accordeur qui détecte en temps réel la note jouée par l'instrument grâce à l'analyse du signal audio capté par le microphone. À partir de cette détection, l'application compare la fréquence entendue à la fréquence de référence de la note cible, puis indique à l'utilisateur comment ajuster la corde : si elle doit être montée ou descendue en tension pour atteindre la justesse.

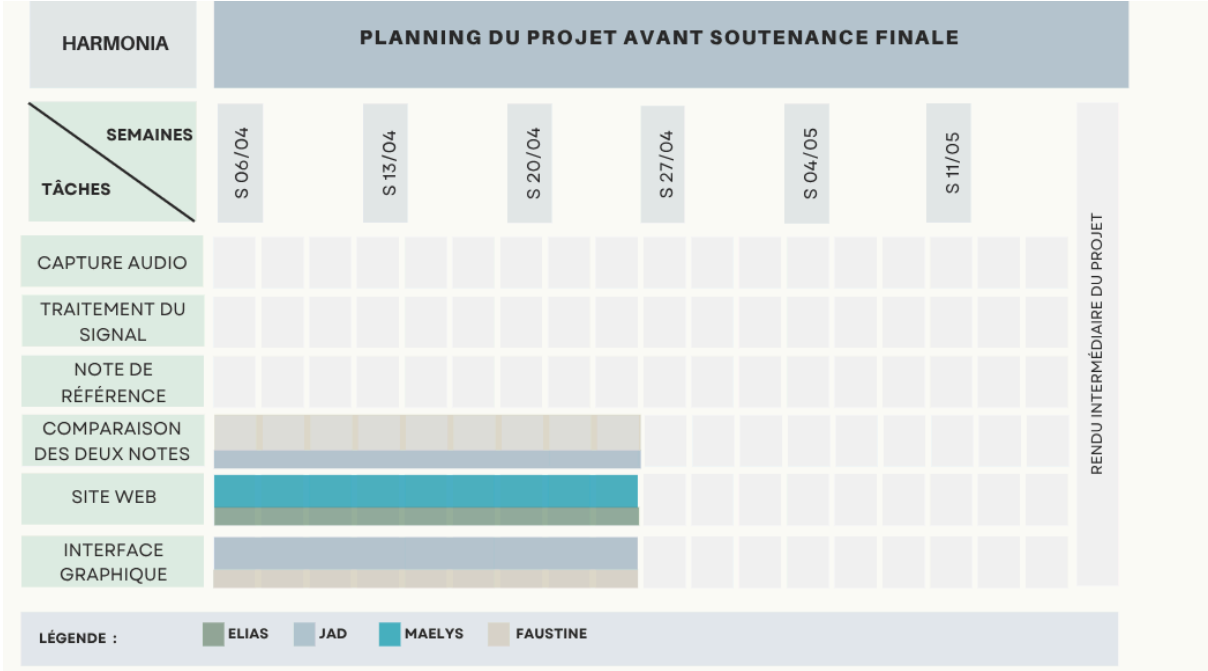
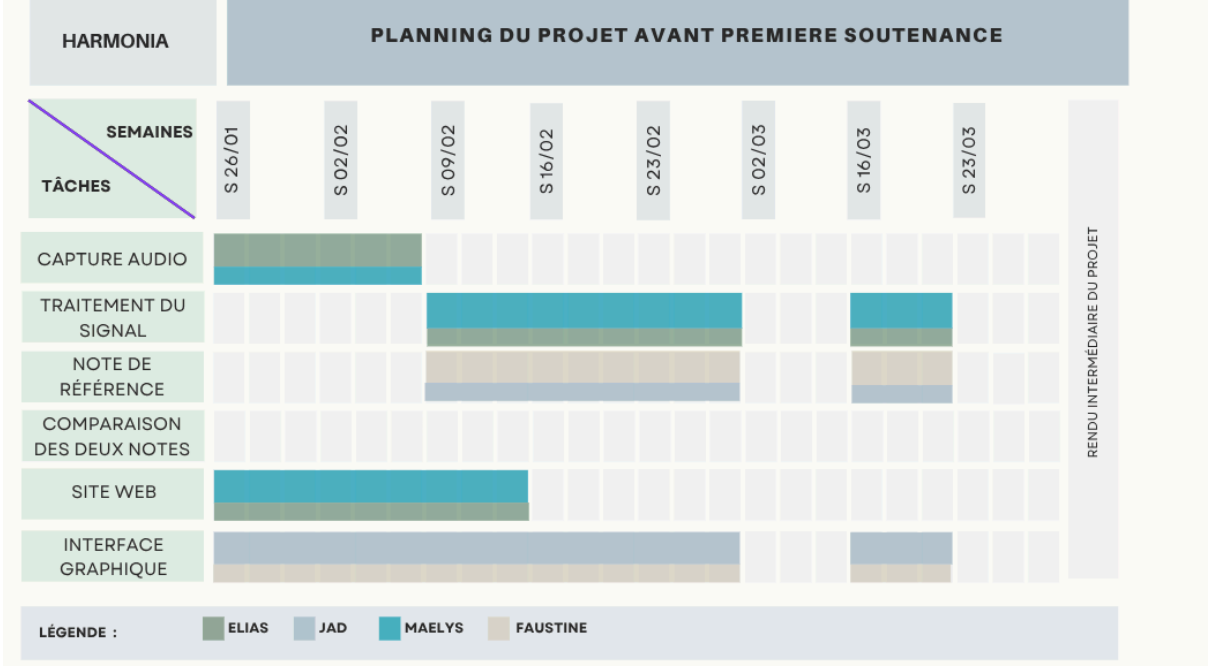
Dans un premier temps, le projet se concentre sur la guitare, instrument parmi les plus joués et donc le plus pertinent comme point de départ. Cependant, TrueTone, au fil du développement, tend à évoluer et veut accueillir d'autres instruments à cordes tels que la basse, le ukulélé, le violon ou le banjo. L'ambition est de faire de TrueTone un accordeur universel, capable de s'adapter aux besoins d'un large panel de musiciens, quel que soit leur instrument.

2 - Planification des tâches

Les différentes tâches à réaliser ont été réparties entre les membres de l'équipe en tenant compte des affinités, des compétences et de la motivation de chacun. Nous avons fait le choix d'une répartition basée sur les préférences individuelles, dans le but de maintenir un niveau d'enthousiasme élevé vis-à-vis du travail à accomplir et de permettre à chaque membre de s'investir dans un domaine qu'il maîtrise déjà ou qui suscite un intérêt particulier.

Voici un diagramme de Gantt résumant les affectations des différentes tâches et l'avancement prévu. Les traits épais correspondent aux titulaires de la tâche et les traits plus fins aux suppléants.

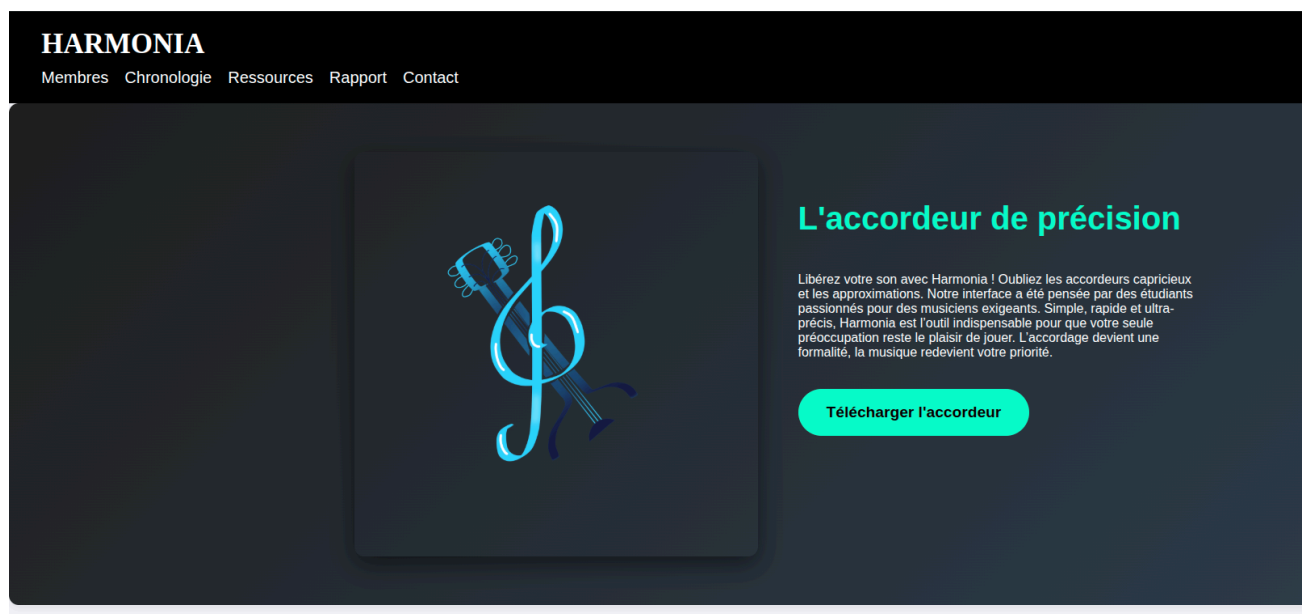
Globalement, aucun retard n'est à noter et il y a même de l'avance sur la tâche de comparaison des deux notes.



III - avancement réel du projet

1) Site web

Le site web présente le projet, l'avancement de celui-ci, l'équipe, un lien vers les deux rapport ainsi que les ressources utilisées et un lien pour télécharger le projet. Pour créer ce site, des difficultés ont été rencontrées, notamment pour l'intégration du menu déroulant permettant de présenter les membres. Finalement, après quelques recherches sur Internet, nous avons compris le fonctionnement et avons pu l'intégrer. Voici un aperçu du site web.



2) Interface Graphique

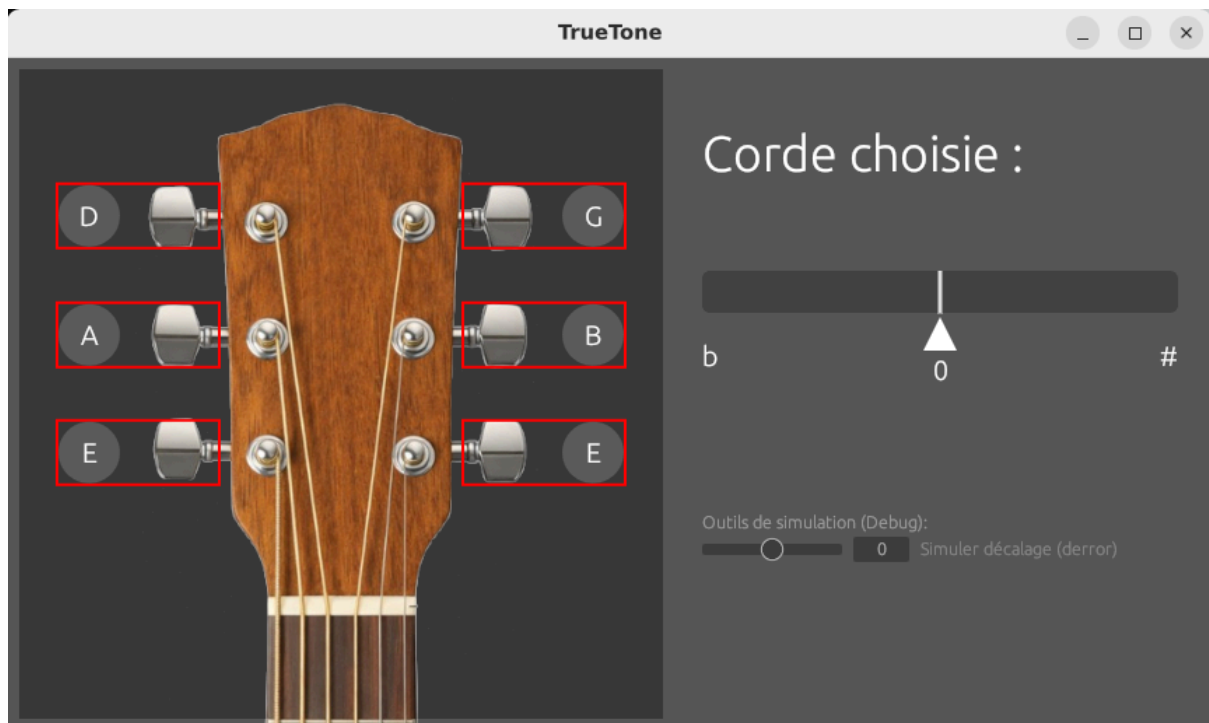
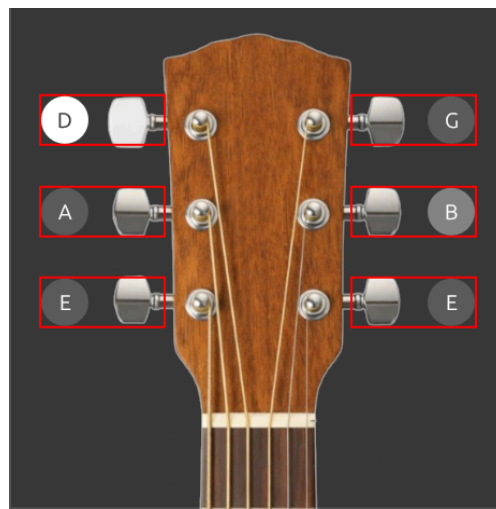


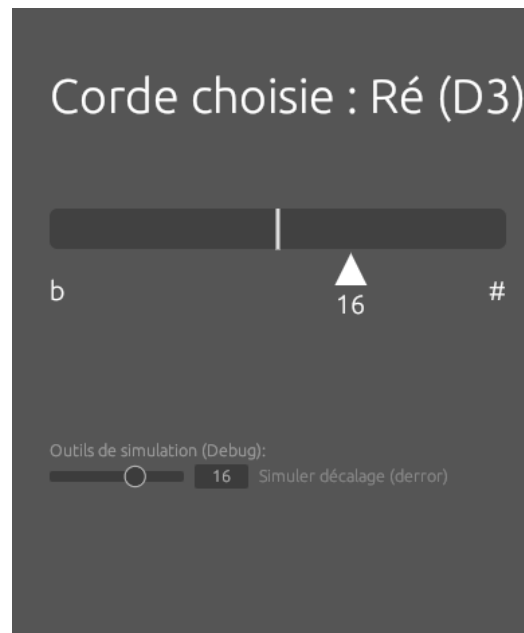
Image de l'application actuelle (en mode debug)

L'application se présente sous la forme d'une fenêtre fixe de 860 × 480 pixels, divisée en deux panneaux :

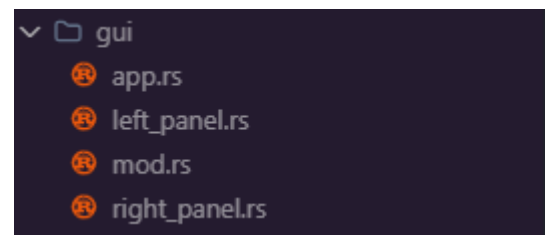
- Le panneau gauche affiche une représentation visuelle de la tête de guitare. Des boutons circulaires sont positionnés à l'emplacement de chaque cheville de la mécanique, permettant à l'utilisateur de sélectionner la corde qu'il souhaite accorder. Lorsqu'une corde est sélectionnée, l'image change pour mettre en évidence la corde correspondante, offrant un retour visuel clair et intuitif.



- Le panneau droit affiche le nom de la corde sélectionnée ainsi qu'une jauge horizontale représentant l'écart de justesse. Un indicateur en forme de flèche se déplace sur cette jauge pour indiquer si la corde est trop basse (b) ou trop haute (#) par rapport à la note cible. Un curseur de simulation a également été intégré à des fins de débogage, permettant de tester le comportement visuel de la jauge.



Sur le plan technique, le code a été structuré de manière modulaire, avec une séparation claire entre la logique de l'application (app.rs), le rendu du panneau gauche (left_panel.rs) et le rendu du panneau droit (right_panel.rs), le tout organisé dans un dossier gui/.



Cependant, l'interface est, pour l'instant, purement visuelle : elle n'est pas encore connectée à un système de détection audio ni à l'analyse fréquentielle pour la correction de note. Le lien entre l'interface graphique et l'analyse du son en temps réel constitue la prochaine étape du développement et permettra à TrueTone de devenir un véritable outil d'accordage fonctionnel.

TrueTone n'en est qu'à ses débuts. La guitare est le premier instrument implémenté, mais, au fil du développement, le projet s'est orienté vers une vision plus large : intégrer d'autres instruments à cordes tels que la basse, le ukulélé ou le violon. Chaque instrument possède son propre accordage de référence et sa propre représentation visuelle, ce qui rendra l'application progressivement plus complète et universelle.

3) Capture audio

La capture audio constitue la première étape de la pipeline de traitement. Son rôle est de récupérer en temps réel le signal provenant du microphone, puis de le transformer en blocs exploitables par le module de traitement du signal.

Pour cela, l'import de la bibliothèque `cpal`, qui permet d'accéder au microphone et de récupérer les échantillons audio de manière portable, est primordiale. La bibliothèque `crossbeam-channel` a aussi été utilisée pour gérer la communication entre la capture audio et le reste du programme. Cela permet de séparer clairement la partie capture du signal et la partie traitement.

Le flux audio reçu est d'abord converti en mono, car l'accordage d'un instrument ne nécessite pas d'information stéréo, et cette conversion simplifie le traitement ultérieur. Les échantillons sont ensuite stockés dans un buffer glissant, ce qui permet de construire des fenêtres de taille fixe.

Le choix de la taille de fenêtre est de 4096 échantillons avec un décalage de 1024 échantillons entre deux fenêtres successives. Ce chevauchement permet d'obtenir un bon compromis entre continuité temporelle, stabilité et réactivité.

Avant l'envoi des données au module suivant, il faut appliquer un léger prétraitement. Tout d'abord, il suffit de retirer la moyenne du signal afin de recentrer les échantillons autour de zéro. Cela permet d'éliminer un éventuel décalage continu du signal, qui pourrait perturber les traitements fréquentiels ultérieurs.

En parallèle, il faut calculer, pour chaque fenêtre, un niveau moyen du signal. Cet indicateur permet d'évaluer l'intensité sonore globale de la fenêtre.

Comme le microphone capte en permanence un bruit de fond, il a été nécessaire d'ajouter une phase de calibration initiale. Pendant les premières fenêtres, le programme mesure le niveau moyen du signal ambiant, puis en déduit un seuil dynamique représentant le niveau minimal au-delà duquel le signal peut être considéré comme significatif. Dans la version actuelle du projet, ce seuil n'est pas encore utilisé pour rejeter les fenêtres les plus faibles, mais il est transmis au reste du pipeline et sert déjà à moduler le prétraitement.

Plus particulièrement, lorsque le niveau d'une fenêtre dépasse ce seuil, une normalisation douce du gain est appliquée afin d'obtenir un signal plus homogène, notamment lorsque la note est jouée faiblement ou que le microphone est éloigné. Cette normalisation reste volontairement limitée afin de ne pas déformer excessivement le signal.

Enfin, les fenêtres audio sont envoyées via un channel borné. Le choix d'un envoi non bloquant (`try_send`) afin d'éviter qu'un ralentissement du traitement ne

bloque la capture était le meilleur. En contrepartie, si la file de transmission est pleine, certaines fenêtres peuvent être abandonnées. Ce choix semble pertinent, car cela se passe en temps réel, donc il est préférable de préserver la continuité de la capture plutôt que de figer le flux audio.

4) Analyse fréquentielle

Cette partie consiste à déterminer la note jouée par l'utilisateur à partir du son récupéré plus tôt dans l'algorithme. Pour cela, c'est le principe de la transformée de Fourier rapide (FFT) qui a été utilisé. Pour améliorer la fiabilité du résultat, un algorithme d'interpolation est ensuite appliqué. Enfin, nous avons codé une fonction qui transforme la fréquence obtenue en note.

Lorsque nous avons fait nos recherches pour savoir quel algorithme utiliser, plusieurs possibilités ont été repérées. Après comparaison, le choix de la FFT était le meilleur. Cet algorithme permet de trouver une note même si le son n'est pas pur, donc ne correspond pas à un signal sinusoïdal. C'est un point important puisqu'une guitare (ou tout instrument à corde) produit des harmoniques. Ce n'était par exemple pas possible avec un algorithme de Zero-Crossing.

Une autre solution aurait pu être un algorithme d'autocorrélation comme l'algorithme de YIN. En effet, les résultats sont un peu plus stables. Mais les résultats de la FFT sont tout de même très bons et cet algorithme possède d'autres avantages. Tout d'abord, la FFT est plus adaptée à un usage en temps réel puisque la complexité est en $O(N \cdot \log N)$, alors que l'algorithme de YIN passe par plusieurs étapes et peut prendre plus ou moins longtemps à trouver le résultat. De plus, en Rust, il existe la crate `rustfft` qui facilite les calculs. Enfin, la FFT permettra plus tard l'ajout des fonctionnalités comme, par exemple, la détection d'accords. En effet, la FFT arrive à détecter quel est l'accord joué par l'utilisateur, ce qui n'est pas le cas des algorithmes d'autocorrélation.

Enfin, la FFT est plus rapide que la transformée de Fourier classique, ce qui permet de gagner en optimisation et de pouvoir faire tourner l'accordeur en temps réel.

5) Comparaison des notes

Pour détecter si le son des notes produit par l'instrument de l'utilisateur est juste ou faux, il est primordial d'importer et de définir des notes de référence dans le programme. Pour cela, il faut se baser sur des notes conventionnelles utilisées à l'international. Grâce à cela, n'importe quel musicien novice ou expert du monde entier pourra comprendre l'instruction donnée.

Les notes utilisées par le logiciel seront donc celles basées sur le système d'accordage du La 440 Hz qui est le standard international. Il y a 6 notes qui correspondent aux 6 cordes d'une guitare. Les notes dans le programme seront notées de la sorte :

- E2, la corde la plus grave, qui est un Mi. Sa fréquence est de 82.41 Hz,
- A2 qui est un La, sa fréquence est de 110.0 Hz,
- D3 qui est un Ré et qui a une fréquence de 143.83 Hz,
- G3 qui est un Sol et qui a une fréquence de 196.0 Hz,
- B3, un Si, qui a une fréquence de 246.94 Hz,
- E4, la corde la plus aiguë, qui est un Mi et qui a une fréquence de 329.63 Hz.

Les chiffres à côté des notes correspondent aux différents octaves.

Une fois que ces notes sont implémentées dans le code, il suffit de faire la partie de comparaison. Celle-ci reprend simplement la note enregistrée lors de la capture audio puis la compare avec les notes de référence. La note de référence choisie par rapport à la note enregistrée est celle qui possède le plus faible écart en hertz puis le programme indique à l'utilisateur soit si sa note est trop grave par rapport à celle accordée, soit si elle est trop aiguë, soit si elle est correcte. Le programme indiquera à l'utilisateur que sa note est accordée à partir du moment où l'écart entre les deux notes est inférieur à 0.5 Hz.

6) Mise en commun et architecture temps réel

La dernière étape du développement a consisté à relier l'ensemble des modules indépendants — capture audio, analyse fréquentielle, comparaison de notes et interface graphique — en un système cohérent et fonctionnel en temps réel.

Architecture globale

Le projet est structuré en plusieurs modules distincts :

- input.rs : capture du signal microphone via cpal
- analyser.rs : analyse fréquentielle par FFT et HPS
- comparaison.rs : comparaison de la fréquence détectée avec les notes de référence
- notes.rs : définition des notes cibles de la guitare
- gui/ : interface graphique (app.rs, left_panel.rs, right_panel.rs)
- main.rs : point d'entrée, orchestration des threads

Le thread audio

Au démarrage de l'application, un thread dédié est lancé depuis main.rs. Ce thread prend en charge l'intégralité du pipeline de traitement du signal, de la capture jusqu'à la fréquence finale communiquée au GUI.

Initialisation dynamique de l'analyseur

Une subtilité importante de cette version est que le FrequencyAnalyzer n'est pas initialisé avec un taux d'échantillonnage fixe. Il est créé dynamiquement lors de la réception du premier chunk audio, en utilisant le sample rate réel.

Cela permet à TrueTone de s'adapter automatiquement à n'importe quel périphérique audio, qu'il fonctionne à 44 100 Hz, 48 000 Hz ou toute autre fréquence d'échantillonnage supportée.

La fréquence n'est calculée que si le niveau sonore du chunk dépasse le seuil calibré dynamiquement par input.rs. Ce seuil est estimé lors des premières frames audio, en mesurant le bruit ambiant moyen, puis mis à l'échelle par un facteur de marge :

Si le niveau est trop faible, la fréquence est directement mise à None, ce qui évite d'afficher des notes parasites dues au bruit de fond.

Lissage exponentiel avec détection de saut

La fréquence brute détectée peut varier brusquement d'une frame à l'autre, notamment lors de l'attaque d'une note ou en cas de détection erronée d'un harmonique. Un lissage exponentiel (EMA) est appliqué pour stabiliser l'affichage, avec un mécanisme de réinitialisation en cas de saut de fréquence trop important.

Le coefficient $\alpha = 0.35$ a été choisi par tests successifs : il offre une bonne réactivité à l'attaque des notes tout en lissant les variations parasites entre deux frames stables. Les seuils 1.8 et 0.55 correspondent approximativement à un écart d'une octave vers le haut ou vers le bas, ce qui permet de distinguer un changement de note réel d'une erreur de détection.

Communication avec le GUI

La fréquence lissée transite entre les deux threads grâce à un Arc<Mutex<Option<f32>>> créé dans main.rs et cloné avant le lancement du thread audio. Une référence est passée au thread audio (shared_freq_audio), l'autre est transmise à l'application graphique (shared_freq).

Lecture dans le GUI

À chaque frame de rendu, app.rs tente de lire la fréquence partagée via try_lock, une variante non bloquante du verrou. Cela évite de figer l'interface si le thread audio est en train d'écrire au même instant. La fréquence lue est ensuite utilisée pour calculer l'écart en cents par rapport à la note cible de la corde sélectionnée, grâce à la fonction cents_deviation de comparaison.rs. La valeur d'erreur est transmise au panneau droit (right_panel.rs), qui l'utilise

pour positionner l'indicateur en forme de flèche sur la jauge horizontale. Un appel à `ctx.request_repaint()` force le rafraîchissement continu de l'interface, assurant une mise à jour en temps réel.

Affichage dans l'interface

Le panneau gauche (`left_panel.rs`) permet à l'utilisateur de sélectionner la corde à accorder en cliquant sur les chevilles de la représentation visuelle de la guitare. Le panneau droit (`right_panel.rs`) affiche en temps réel la note détectée ainsi que la position de la flèche sur la jauge, indiquant si la corde est trop grave (♭) ou trop aiguë (#).

7) Difficultés rencontrées

Première soutenance

Pour l'interface graphique, la principale difficulté rencontrée au cours du développement a été le positionnement précis des boutons sur l'image de la tête de guitare. Chaque bouton devait correspondre exactement à l'emplacement visuel des chevilles sur l'image, ce qui s'est révélé relativement complexe, car les coordonnées dépendent à la fois de la taille de l'image, de son centrage dans le panneau et des décalages propres à chaque cheville. Les hitbox, c'est-à-dire les zones cliquables associées à chaque bouton, devaient également être ajustées individuellement pour être ni trop petites ni trop grandes, afin de garantir une interaction fluide et intuitive. Pour corriger ces positions, il a été nécessaire d'afficher temporairement les contours des hitbox en rouge directement sur l'interface, permettant de visualiser en temps réel l'écart entre la zone cliquable et la position réelle de la cheville sur l'image.

Lors de l'implémentation de l'analyse fréquentielle, la première difficulté était de bien comprendre le principe de la FFT et de savoir comment bien l'implémenter. Il a donc fallu faire différentes recherches sur le fonctionnement de l'algorithme ainsi que sur ceux qui étaient déjà codés en Rust et qui étaient utilisables.

C'était ensuite compliqué de savoir comment tout organiser, quelle structure et quelles variables nous avons besoin et comment optimiser le plus possible le code afin de pouvoir l'utiliser en temps réel. Finalement, la conclusion était qu'une partie des calculs était faisable, comme les allocations mémoires, avant la boucle qui récupère les informations sur le son. Cela évite de refaire une allocation mémoire à chaque tour de boucle.

Enfin, un autre problème a été rencontré : l'algorithme ne détecte pas lorsqu'aucun son n'est joué. Cela est dû au son ambiant autre que l'instrument. L'ordinateur détecte toujours du son, même minime. Nous avons tenté de mettre un seuil minimum, mais pour l'instant cela ne fonctionne pas ; donc, même lorsque l'utilisateur ne joue pas, l'algorithme calcule une fréquence et la note correspondante. Nous réglerons donc ce problème pour le rendu final du projet.

Pour ce qui est de la capture audio, la première difficulté a concerné la gestion des périphériques audio. Selon la machine utilisée, le microphone ne propose pas exactement les mêmes configurations. Il a donc fallu mettre en place une sélection automatique de configuration, en privilégiant un signal mono, un format flottant (f32) et une fréquence d'échantillonnage la plus proche possible de 44100 Hz. En pratique, certains systèmes imposent néanmoins 48000 Hz, ce qui a nécessité de rendre le code compatible avec plusieurs cas.

Un autre problème important a été la présence constante d'un bruit de fond. Même lorsqu'aucune note n'est jouée, le microphone renvoie toujours un signal non nul. Une phase de calibration permettant d'estimer le bruit ambiant et de calculer un seuil a donc été ajoutée. Cette solution améliore déjà la robustesse du pipeline, même si elle n'est pas encore exploitée jusqu'au bout dans cette première version.

Enfin, le fonctionnement en temps réel a dû être pris en compte attentivement. La capture audio doit rester continue, même si le traitement prend du retard. Pour cela, il a fallu choisir un mécanisme de transmission non bloquant. C'est un compromis : il peut conduire à la perte de certaines fenêtres lorsque la file est pleine, mais il garantit que la capture ne se bloque pas.

Soutenance finale

Lors de la phase finale du projet, plusieurs difficultés sont apparues lorsque nous avons commencé à tester l'accordeur sur une guitare réelle. Les premiers tests réalisés avec des fréquences générées artificiellement fonctionnaient correctement, mais le comportement du programme était moins fiable avec un vrai instrument. Il a donc été nécessaire d'améliorer plusieurs parties du pipeline de traitement.

Correction de la fréquence d'échantillonnage

Un premier problème venait du fait que l'analyse fréquentielle utilisait initialement une fréquence d'échantillonnage fixe de 44100 Hz. Or, selon le microphone ou le système d'exploitation, certains périphériques audio fonctionnent en 48000 Hz.

Comme la conversion entre les indices de la FFT et les fréquences dépend directement du sample rate, cela provoquait des erreurs dans les fréquences détectées. Nous avons donc modifié le programme afin d'utiliser automatiquement la fréquence d'échantillonnage réelle fournie par le périphérique audio via cpal. Cette valeur est désormais transmise jusqu'à l'analyseur fréquentiel, ce qui rend les résultats beaucoup plus précis et cohérents entre différentes machines.

Amélioration de la détection fréquentielle

Une autre difficulté importante concernait la détection des notes sur une guitare réelle. La première version du programme détectait simplement le pic spectral le plus élevé après la FFT. Cette méthode fonctionne correctement sur des signaux purs, mais une guitare produit un signal beaucoup plus complexe contenant une fréquence fondamentale ainsi que plusieurs harmoniques.

Dans certains cas, l'algorithme détectait alors une harmonique au lieu de la fréquence fondamentale. Par exemple, la corde de Mi aigu pouvait parfois être détectée autour de 164 Hz au lieu de 329 Hz, ce qui correspond à une erreur d'octave.

Pour corriger cela, une logique supplémentaire a été ajoutée après la FFT. Le programme ne se contente plus de prendre le pic le plus fort : il compare également plusieurs hypothèses autour de la fréquence détectée, notamment la fréquence elle-même, sa moitié et son double. Ensuite, il sélectionne la valeur la plus cohérente avec les fréquences réelles des six cordes de guitare. Cette approche permet de réduire fortement les erreurs d'octave et de rendre la détection plus robuste face aux harmoniques.

Ajustement du découpage en fenêtres

Nous avons également rencontré des problèmes de réactivité. Une version intermédiaire utilisait des fenêtres non chevauchantes, ce qui rendait l'accordeur moins fluide et moins réactif.

Le `hop_size` a donc été réajusté à 1024 échantillons pour des fenêtres de 4096 échantillons. Les fenêtres se chevauchent désormais à nouveau, ce qui améliore la continuité temporelle et permet un suivi plus fluide des variations de fréquence.

Réglage des seuils et calibration

Pendant les tests, nous avons remarqué qu'il fallait parfois jouer les cordes très fort pour que l'accordeur détecte correctement la note. Le problème venait des seuils utilisés pour distinguer une vraie note du bruit ambiant.

Le programme réalise une phase de calibration au démarrage afin d'évaluer le niveau moyen du bruit de fond. À partir de cette mesure, un seuil dynamique est calculé. Nous avons ensuite ajusté ce système afin de rendre la détection plus sensible, tout en évitant que le bruit ambiant soit constamment interprété comme une note jouée.

Stabilisation des fréquences affichées

Enfin, les fréquences détectées variaient légèrement d'une fenêtre à l'autre, ce qui rendait l'affichage instable. Pour améliorer cela, un lissage exponentiel de type EMA a été conservé et ajusté.

Le principe est de calculer une moyenne pondérée entre l'ancienne fréquence et la nouvelle fréquence détectée. Cela réduit les variations brusques tout en gardant une

bonne réactivité. Grâce à ce lissage, l'accordeur affiche désormais des valeurs plus stables et plus agréables à utiliser en temps réel.

IV - Bibliographie

Site utilisé pour la transformée de Fourier rapide :

- <https://www.f-legrand.fr/scidoc/docmml/numerique/tfd/fft3/fft3.html>
- https://svantek.com/fr/academie/transformee-de-fourier-rapide-fft/#elementor-toc_heading-anchor-2

Bibliothèque nécessaire pour l'interface graphique :

- egui – *An immediate GUI library for Rust.*
<https://docs.rs/egui/latest/egui/>
- eframe – *Framework for egui apps.*
<https://docs.rs/eframe/latest/eframe/>

Site utilisé pour la capture audio:

- [cpal - Rust](#)
- [crossbeam_channel - Rust](#)
- [The Joy of the Unknown: Exploring Audio Streams with Rust and Circular Buffers - DEV Community](#)